



طراحی تفصیلی رابط‌های برنامه‌نویسی (API)

سامانه یکپارچه آموزش آنلاین مدارس

فاز پنجم: طراحی تفصیلی سیستم | ارزیابی اصول | SOLID الگوهای طراحی GoF

مقدمه و اهداف طراحی API

در فاز طراحی تفصیلی، معماری مفهومی سامانه (کلاینت-سرور، لایه‌ای، خدمت‌گرا و رویدادمحور) به مشخصات فنی قابل پیاده‌سازی تبدیل می‌شود. رابط‌های برنامه‌نویسی (API) تنها مسیر رسمی ارتباط میان سرویس‌های مستقل سامانه هستند و کیفیت طراحی آن‌ها مستقیماً بر امنیت، کارایی، توسعه‌پذیری و یکپارچگی کل سامانه اثرگذار است.

قرارداد ارتباطی استاندارد میان تمامی سرویس‌ها



کاهش وابستگی میان زیرسیستم‌ها (Loose Coupling)



توسعه مستقل هر سرویس توسط تیم‌های مختلف



قابلیت استفاده مجدد در ماژول‌ها و کلاینت‌های گوناگون



پشتیبانی از نسخه‌های آینده بدون ناسازگاری



اصول طراحی رابط‌های برنامه‌نویسی



استقلال سرویس‌ها

هر سرویس مالک دامنه مشخص خود بوده و از طریق رابط عمومی با دیگران ارتباط برقرار می‌کند.



قرارداد ثابت (Contract First)

هر API پیش از پیاده‌سازی، به‌صورت یک قرارداد رسمی شامل Endpoint، ورودی و خروجی تعریف می‌شود.



ارتباط بدون وضعیت (Stateless)

هر درخواست مستقل بوده و سرور وضعیتی از نشست کاربر نگهداری نمی‌کند.



طراحی مبتنی بر منبع (Resource-Oriented)

API‌ها حول منابعی مانند کاربران، کلاس‌ها و آزمون‌ها طراحی می‌شوند.



نسخه‌بندی (Versioning)

تمامی API‌ها شماره نسخه دارند تا تغییرات ناسازگار، سرویس‌های موجود را مختل نکنند.

استانداردهای ارتباطی و امنیت

استانداردهای فنی ارتباط

معماری API	RESTful API
پروتکل ارتباطی	HTTPS
قالب تبادل داده	JSON
نوع رمزگذاری	UTF-8
احراز هویت	JWT Bearer Token
ارتباط بلادرنگ	WebSocket
رویدادهای ناهمزمان	Event Bus

اصول امنیت APIها

دسترسی صرفاً از طریق پروتکل HTTPS برای محرمانگی داده‌ها



احراز هویت مبتنی بر JWT و کنترل دسترسی مبتنی بر نقش (RBAC)



اعتبارسنجی تمامی ورودی‌ها پیش از پردازش



محدودسازی نرخ درخواست‌ها (Rate Limiting) برای جلوگیری از سوءاستفاده



ثبت رویدادهای امنیتی و مدیریت نشست‌های منقضی‌شده



سرویس‌های سامانه — نمای کلی

معماری خدمت‌گرا (SOA) سامانه از ده سرویس مستقل تشکیل شده که هر یک مالک حوزه مشخصی از داده و منطق کسب‌وکار است.



احراز هویت

Authentication



مدیریت کاربران

User Mgmt



مدیریت کلاس‌ها

Class Mgmt



مدیریت محتوا

Content Mgmt



مدیریت تکالیف

Assignment



آزمون‌های آنلاین

Examination



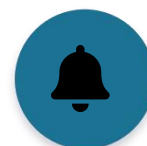
حضور و غیاب

Attendance



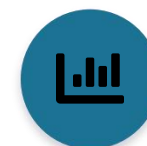
پیام‌رسان داخلی

Messaging



اعلان‌ها

Notification



گزارش‌گیری و تحلیل

Reporting

نمونه سرویس: احراز هویت (Authentication)

POST /api/v1/auth/login

```
{
  "nationalCode": "1234567890",
  "password": "*****"
}

{
  "success": true,
  "data": { "accessToken": "JWT...", "expiresIn": 3600, "role": "Teacher" }
}
```

الزامات امنیتی کلیدی

- رمزنگاری گذرواژه با Bcrypt
- توکن Refresh مستقل از Access Token
- قفل موقت حساب پس از چند تلاش ناموفق
- ثبت کامل تلاش‌های ورود ناموفق

Endpoint های سرویس

/auth/login	POST	ورود کاربر
/auth/logout	POST	خروج کاربر
/auth/refresh	POST	تمدید توکن
/auth/forgot-password	POST	درخواست بازیابی رمز
/auth/reset-password	POST	تغییر رمز عبور
/auth/validate	GET	اعتبارسنجی توکن

قالب استاندارد پاسخ API

تمامی سرویس‌ها پاسخ‌های خود را با قالبی یکنواخت ارائه می‌کنند تا پردازش پاسخ در تمام کلاینت‌ها ساده و یکپارچه باشد.



پاسخ موفق

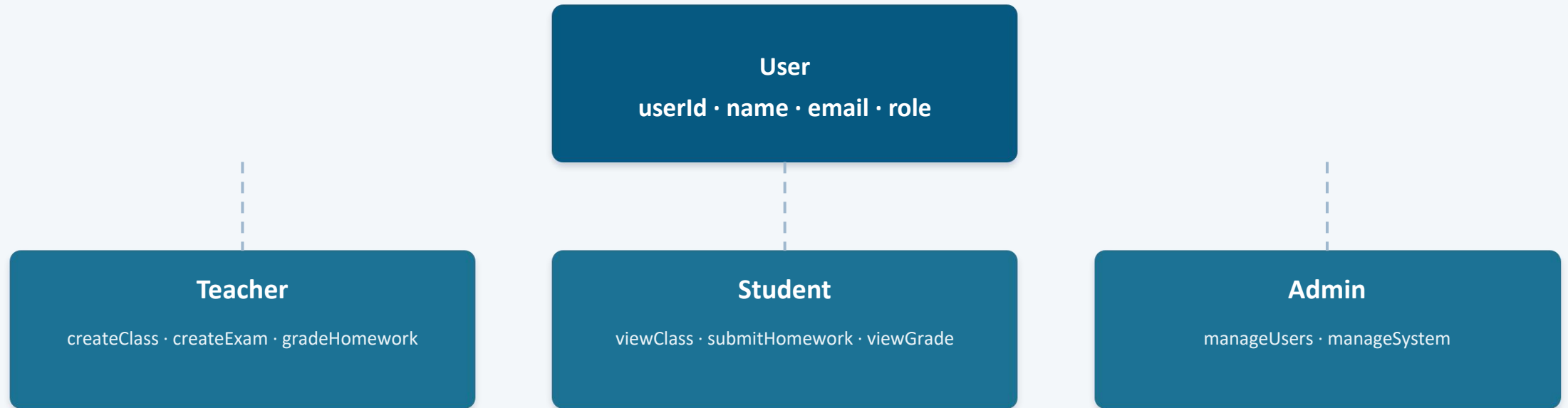
```
{
  "success": true,
  "message": "Operation completed
    successfully.",
  "data": { ... }
}
```



پاسخ ناموفق

```
{
  "success": false,
  "errorCode": "AUTH_401",
  "message": "Authentication
    failed."
}
```

معماری شیء‌گرا — سلسله‌مراتب کاربران



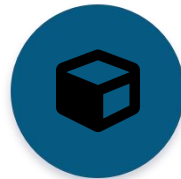
بر اساس اصل جایگزینی لیسکوف (LSP)، هر یک از کلاس‌های **Teacher**، **Student** و **Admin** می‌توانند بدون ایجاد اختلال در رفتار سیستم، جایگزین کلاس پایه **User** شوند. تنها در عملیات وابسته به نقش (مانند دسترسی به نمرات) نیاز به بررسی نقش در زمان اجرا وجود دارد.

ارزیابی طراحی بر اساس اصول SOLID

۱ از ۳

SRP — Single Responsibility Principle

اصل مسئولیت واحد



سیستم به ماژول‌های مستقل تقسیم شده است. مدیریت کاربران، کلاس‌های آموزشی، تکالیف ، آزمون‌ها و پیام‌رسان — هر ماژول یک مسئولیت مشخص دارد.

این تفکیک باعث شده تغییر در یک بخش (مثلاً آزمون‌ها) بر سایر بخش‌ها اثر نگذارد؛ اما در نسخه اولیه، «دانش‌پورد دانش‌آموز» چند مسئولیت هم‌زمان داشت و نیازمند تفکیک بیشتر است.



OCP — Open/Closed Principle

اصل باز/بسته



سیستم آزمون‌ها و تکالیف به‌صورت ماژولار طراحی شده‌اند تا امکان افزودن نوع سؤال جدید (تستی، تشریحی، آپلود فایل) بدون تغییر ساختار اصلی فراهم شود.

با توسعه کلاس `QuestionType` می‌توان انواع جدید سؤال را بدون تغییر در منطق اصلی آزمون اضافه کرد. ساختار سیستم تا حد زیادی با OCP سازگار است.

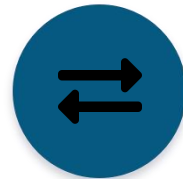


ارزیابی طراحی بر اساس اصول SOLID

۲ از ۳

LSP — Liskov Substitution Principle

اصل جایگزینی لیسکوف



ساختار کاربران به صورت ارث‌بری طراحی شده: کلاس پایه User و زیرکلاس‌های Student، Teacher و Admin.

تمام زیرکلاس‌ها می‌توانند بدون اختلال در سیستم جایگزین کلاس پایه شوند. نکته: در عملیاتی مانند دسترسی به نمرات، بررسی نقش در زمان اجرا لازم است که در طراحی ضعیف می‌تواند LSP را نقض کند.



ISP — Interface Segregation Principle

اصل تفکیک رابط‌ها



هیچ کاربری نباید مجبور به پیاده‌سازی متدهایی شود که به آن نیازی ندارد. رابط‌ها به صورت جداگانه تعریف شده‌اند: IAuthService، IClassService، IChatService، IExamService.

این تفکیک باعث شده معلمان، دانش‌آموزان و مدیران فقط با سرویس‌های مرتبط با نقش خود در تعامل باشند و از وابستگی غیرضروری جلوگیری شود.

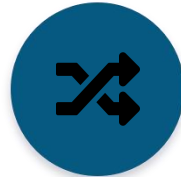


ارزیابی طراحی بر اساس اصول SOLID

۳ از ۳

DIP — Dependency Inversion Principle

اصل معکوس سازی وابستگی



ماژول های سطح بالا نباید به ماژول های سطح پایین وابسته باشند. فرانت اند از طریق REST API Interface و تزریق وابستگی (Dependency Injection) با بک اند ارتباط برقرار می کند. Dashboard → ClassService → API → Database.

این ساختار باعث شده تغییر در دیتابیس یا بک اند تأثیری بر فرانت اند نداشته باشد و لایه بندی سیستم مستحکم بماند.



جمع بندی کلی ارزیابی SOLID

افزایش مقیاس پذیری سیستم



کاهش وابستگی میان ماژول ها



افزایش قابلیت تست پذیری



بهبود نگهداری و توسعه آینده



با این حال، بخش هایی مانند داشبوردهای ترکیبی و منطق های چندمسئولیتی نیازمند بازطراحی و Refactoring هستند تا انطباق کامل با اصول SOLID حاصل شود.

الگوی طراحی Observer

مدیریت اعلان‌ها و اطلاع‌رسانی رویدادهای سیستم



توجیه انتخاب

کلاس‌های دامنه مانند Assignment، Exam و ClassSession تنها رویداد خود را منتشر می‌کنند و هیچ وابستگی مستقیمی به سیستم اعلان ندارند؛ برای نمونه، ایجاد یک تکلیف جدید صرفاً یک رویداد منتشر می‌کند و NotificationEventListener مستقل از آن، اعلان مرتبط را برای دانش‌آموزان ارسال می‌کند.

کاهش وابستگی بین کلاس‌ها (Loose Coupling)



افزایش قابلیت توسعه سیستم



امکان افزودن کانال‌های اطلاع‌رسانی جدید (پیامک، ایمیل)



سازگار با اصل Open/Closed (OCP)



الگوی طراحی Strategy

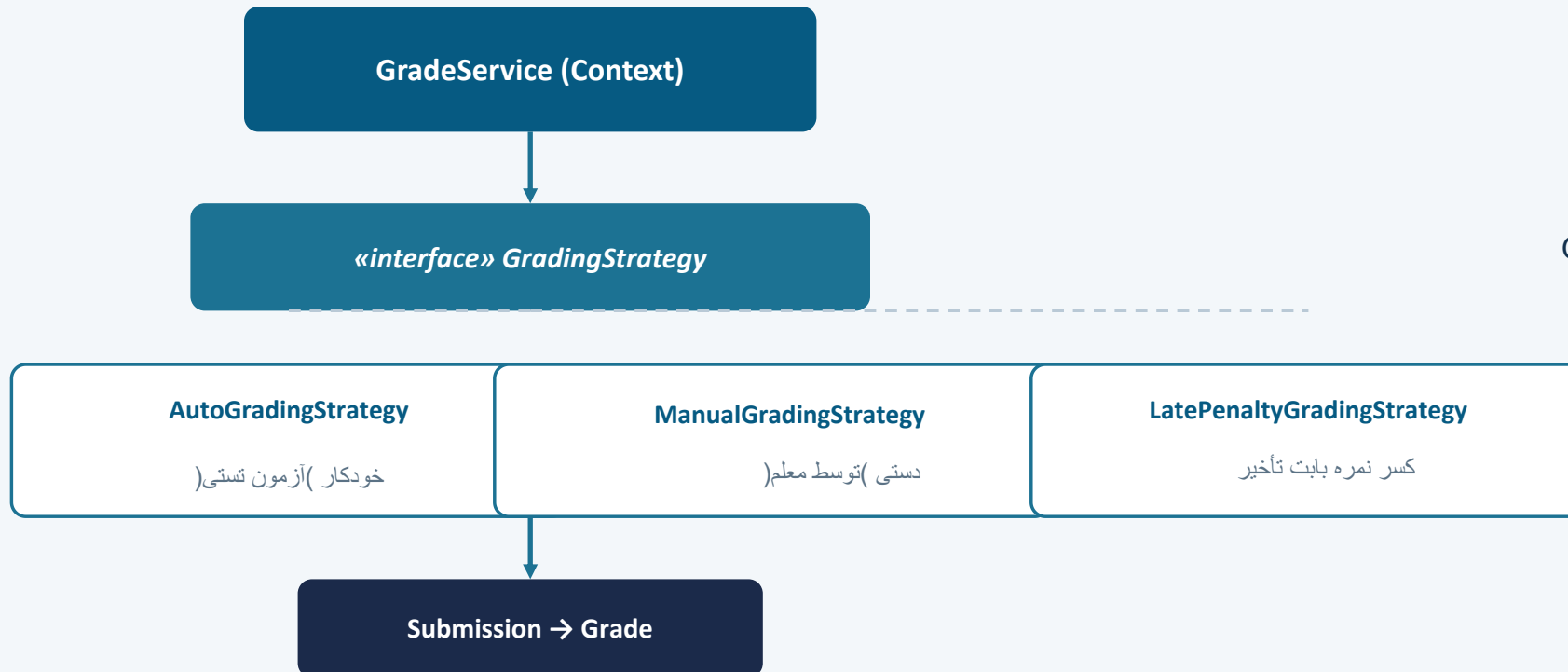
مدیریت فرآیند نمره‌دهی و ارزیابی پاسخ‌های دانش‌آموزان

توجیه انتخاب

جداسازی منطق نمره‌دهی از کلاس‌های دامنه

جلوگیری از شرط‌های پیچیده در GradeService

سازگار با اصول DIP و OCP



جمع‌بندی الگوهای طراحی GoF

الگو	محل استفاده	کلاس‌های مرتبط	دلیل انتخاب
Observer	مدیریت اعلان‌ها — Assignment, Exam, ClassSession	DomainEvent, EventPublisher, EventListener, NotificationEventListener, NotificationService, Notification	کاهش وابستگی بین کلاس‌ها و اطلاع‌رسانی خودکار
Strategy	نمردهی و ارزیابی پاسخ‌ها	GradeService, Submission, GradingStrategy, AutoGradingStrategy, ManualGradingStrategy, LatePenaltyGradingStrategy, Grade	جداسازی الگوریتم‌های نمردهی و افزایش قابلیت توسعه

الگوی Observer مدیریت رویدادها و اطلاع‌رسانی خودکار را بر عهده دارد و الگوی Strategy امکان استفاده از روش‌های مختلف نمردهی را فراهم می‌کند؛ هر دو الگو با اصول DIP و OCP از مجموعه اصول SOLID سازگار هستند.

جمع‌بندی نهایی

معماری خدمت‌گرا و RESTful



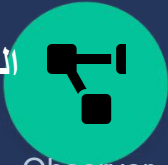
ده سرویس مستقل با قرارداد API استاندارد و نسخه‌بندی‌شده

انطباق با اصول SOLID



طراحی ماژولار، قابل توسعه و کم‌وابسته با نیاز به بهبود در بخش‌های ترکیبی

الگوهای طراحی GoF



Observer برای اعلان‌ها و Strategy برای نمردهی، هر دو سازگار با OCP/DIP

امنیت و مقیاس‌پذیری



JWT، RBAC، HTTPS و معماری رویدادمحور برای عملکرد پایدار سامانه